

```

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>
#include <string.h>
#include <stdbool.h>

/* Global Initialized Variables (.data segment) */
int g_initialized_status = 1;
const char *g_file_path = "shared_file";
size_t g_total_bytes_processed = 0;

/* Global Uninitialized Variables (.bss segment) */
pthread_mutex_t g_sync_mutex;
pthread_cond_t g_data_cond;
char g_shared_transfer_char;

/* Synchronization Control Flags (.bss segment - initialized to false by default) */
bool g_has_data = false;
bool g_stream_finished = false;

/* Reader Thread Routine */
void *reader_thread_func(void *arg) {
    /* Reader Thread Local Variables (Stack Frame) */
    int local_fd = -1;
    ssize_t local_bytes_read = 0;
    char *read_buf = NULL;

    /* Allocate 1-byte dynamic buffer on the Heap */
    read_buf = (char *)malloc(sizeof(char));
    if (read_buf == NULL) {
        perror("Reader: Failed to allocate read_buf");
        pthread_exit(NULL);
    }

    /* Open shared_file for reading using Linux System Call open() */
    local_fd = open(g_file_path, O_RDONLY);
    if (local_fd == -1) {
        perror("Reader: Failed to open file");
        free(read_buf);
        pthread_exit(NULL);
    }

    /* Processing Loop: Read byte-by-byte via system call read() */
    while ((local_bytes_read = read(local_fd, read_buf, 1)) > 0) {
        pthread_mutex_lock(&g_sync_mutex);

        /* Wait until the Writer thread consumes the previous byte */
        while (g_has_data) {
            pthread_cond_wait(&g_data_cond, &g_sync_mutex);
        }
    }
}

```

```

    }

    /* Transfer byte to shared global transfer storage */
    g_shared_transfer_char = *read_buf;
    g_has_data = true;

    /* Signal the Writer thread that a new byte is available */
    pthread_cond_signal(&g_data_cond);
    pthread_mutex_unlock(&g_sync_mutex);
}

if (local_bytes_read == -1) {
    perror("Reader: Error occurred during read()");
}

/* Signal stream completion to Writer thread */
pthread_mutex_lock(&g_sync_mutex);
while (g_has_data) {
    pthread_cond_wait(&g_data_cond, &g_sync_mutex);
}
g_stream_finished = true;
pthread_cond_signal(&g_data_cond);
pthread_mutex_unlock(&g_sync_mutex);

/* Cleanup resources */
close(local_fd);
free(read_buf);
pthread_exit(NULL);
}

/* Writer Thread Routine */
void *writer_thread_func(void *arg) {
    /* Writer Thread Local Variables (Stack Frame) */
    int local_processed_count = 0;
    ssize_t local_bytes_written = 0;
    char *write_buf = NULL;

    /* Allocate 1-byte dynamic buffer on the Heap */
    write_buf = (char *)malloc(sizeof(char));
    if (write_buf == NULL) {
        perror("Writer: Failed to allocate write_buf");
        pthread_exit(NULL);
    }

    while (1) {
        pthread_mutex_lock(&g_sync_mutex);

        /* Wait until Reader supplies new data or signals EOF */
        while (!g_has_data && !g_stream_finished) {
            pthread_cond_wait(&g_data_cond, &g_sync_mutex);
        }

        /* Check for termination condition */
        if (!g_has_data && g_stream_finished) {

```

```

        pthread_mutex_unlock(&g_sync_mutex);
        break;
    }

    /* Retrieve byte into dynamic write buffer */
    *write_buf = g_shared_transfer_char;
    g_has_data = false;
    local_processed_count++;

    /* Echo byte to standard output via system call write() */
    local_bytes_written = write(STDOUT_FILENO, write_buf, 1);
    if (local_bytes_written == -1) {
        perror("Writer: Error writing to stdout");
    }

    /* Signal Reader thread that buffer is cleared */
    pthread_cond_signal(&g_data_cond);
    pthread_mutex_unlock(&g_sync_mutex);
}

/* Update global metric */
pthread_mutex_lock(&g_sync_mutex);
g_total_bytes_processed = (size_t)local_processed_count;
pthread_mutex_unlock(&g_sync_mutex);

write(STDOUT_FILENO, "\n", 1);

/* Cleanup resources */
free(write_buf);
pthread_exit(NULL);
}

/* Main Thread Execution Entry Point */
int main(void) {
    /* Main Thread Local Variables (Stack Frame) */
    int main_fd = -1;
    pthread_t reader_tid;
    pthread_t writer_tid;

    const char *payload =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    size_t payload_len = strlen(payload);

    /* Initialize Synchronization Primitives dynamically */
    if (pthread_mutex_init(&g_sync_mutex, NULL) != 0) {
        perror("Main: Mutex initialization failed");
        return EXIT_FAILURE;
    }
    if (pthread_cond_init(&g_data_cond, NULL) != 0) {
        perror("Main: Condition variable initialization failed");
        pthread_mutex_destroy(&g_sync_mutex);
        return EXIT_FAILURE;
    }
}

```

```

/* Step 1: Create shared_file */
printf("Start creating shared_file\n");

main_fd = open(g_file_path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (main_fd == -1) {
    perror("Main: Failed to create file");
    pthread_mutex_destroy(&g_sync_mutex);
    pthread_cond_destroy(&g_data_cond);
    return EXIT_FAILURE;
}

if (write(main_fd, payload, payload_len) == -1) {
    perror("Main: Failed to write payload");
    close(main_fd);
    pthread_mutex_destroy(&g_sync_mutex);
    pthread_cond_destroy(&g_data_cond);
    return EXIT_FAILURE;
}

if (close(main_fd) == -1) {
    perror("Main: Failed to close file");
    pthread_mutex_destroy(&g_sync_mutex);
    pthread_cond_destroy(&g_data_cond);
    return EXIT_FAILURE;
}

printf("Finished creating shared_file\n");

/* Step 2: Spawn Reader Thread */
printf("Start creating the reader thread\n");
if (pthread_create(&reader_tid, NULL, reader_thread_func, NULL) != 0) {
    perror("Main: Failed to create reader thread");
    return EXIT_FAILURE;
}
printf("created the reader thread\n");

/* Step 3: Spawn Writer Thread */
printf("Start creating the writer thread\n");
if (pthread_create(&writer_tid, NULL, writer_thread_func, NULL) != 0) {
    perror("Main: Failed to create writer thread");
    return EXIT_FAILURE;
}
printf("created the writer thread\n");

/* Step 4: Join Secondary Threads */
pthread_join(reader_tid, NULL);
pthread_join(writer_tid, NULL);

/* Clean up global synchronization primitives */
pthread_mutex_destroy(&g_sync_mutex);
pthread_cond_destroy(&g_data_cond);

return EXIT_SUCCESS;
}

```

